Apache Hive

- Hive is built on the top of Hadoop and is used to process structured data in Hadoop.
- Hive was developed by Facebook.
- It provides various types of querying language which is frequently known as Hive Query Language.
- Apache Hive is a data warehouse and which provides an SQL-like interface between the user and the Hadoop distributed file system (HDFS) which integrates Hadoop.

Features of Hive

- The best feature is it offers data summarization, query, and analysis in much easier manner.
- However, to process data without actually storing in HDFS, Hive supports external tables.
- Also, to improve performance it supports partitioning of data at the level of tables.
- While it comes to optimizing logical plans, Hive has a rule-based optimizer available.
- It allows different storage types such as plain text, RCFile, and HBase.
- It uses indexing to accelerate queries.
- Hive is scalable, familiar, and extensible in nature.
- For working with HiveQL Knowledge of basic SQL query is enough. We don't need any knowledge of programming language.
- By using Hive, it is possible to process structured data in Hadoop.
- Hive makes Querying very simple, as same as SQL.
- By using Hive, it is possible to run Ad-hoc queries for the data analysis

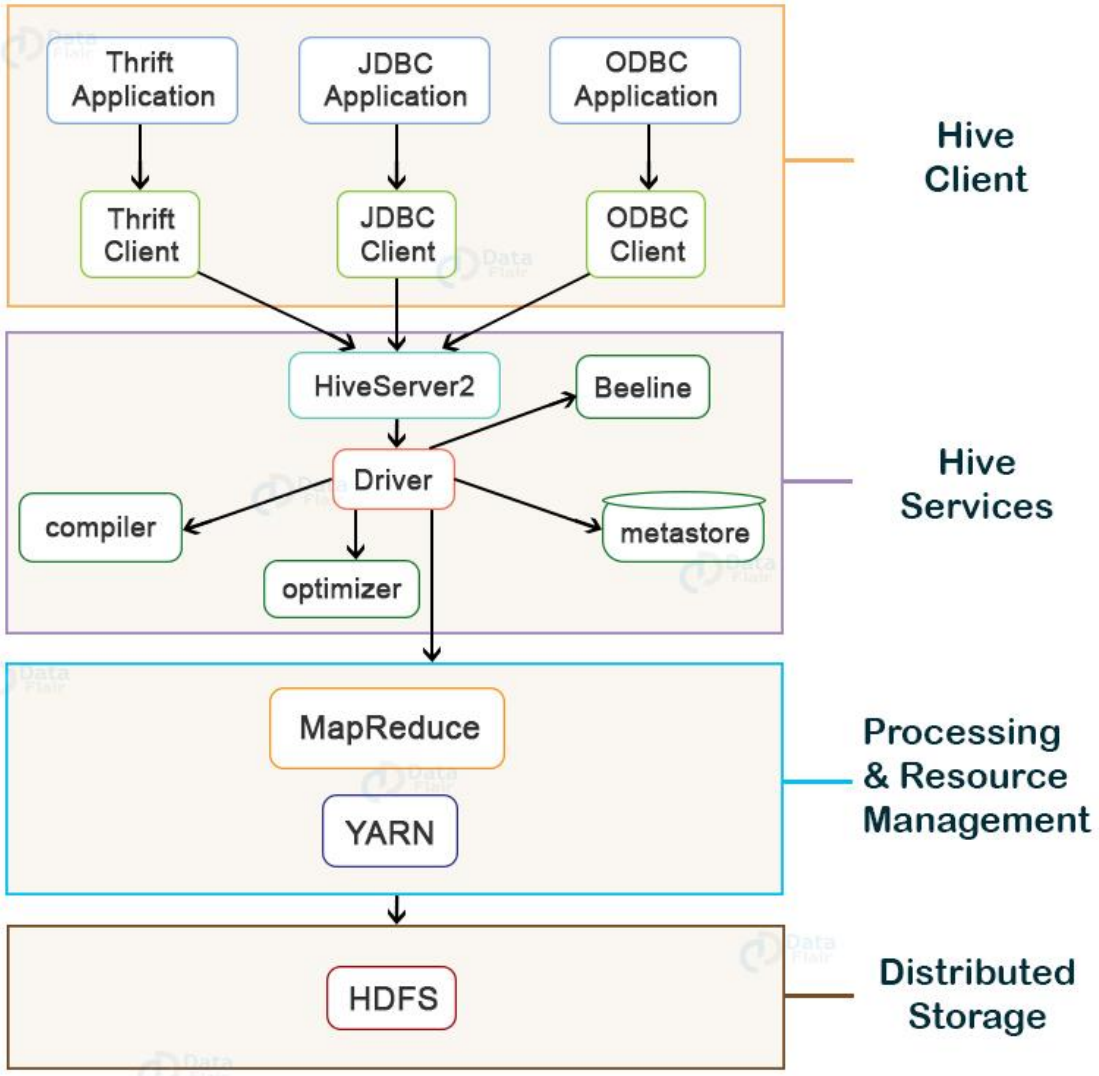
Limitations of Hive

- Hive is not capable of handling real-time data.
- It is not designed for online transaction processing.
- Hive queries contain high latency.

Differences between Hive and Pig

Hive	Pig
Hive is commonly used by Data Analysts.	Pig is commonly used by programmers.
It follows SQL-like queries.	It follows the data-flow language.
It can handle structured data.	It can handle semi-structured data.
It works on server-side of HDFS cluster.	It works on client-side of HDFS cluster.
Hive is slower than Pig.	Pig is comparatively faster than Hive.

Hive Architecture



Hive Architecture & Its Components

The major components of Apache Hive are:

1. **Hive Client**
2. **Hive Services**
3. **Processing and Resource Management**
4. **Distributed Storage**

- Hive Client

Hive supports applications written in any language like Python, Java, C++, Ruby, etc. using JDBC, ODBC, and Thrift drivers, for performing queries on the Hive. Hence, one can easily write a hive client application in any language of its own choice.

Hive clients are categorized into three types:

1. Thrift Clients

The Hive server is based on Apache Thrift so that it can serve the request from a thrift client.

2. JDBC client

Hive allows for the Java applications to connect to it using the JDBC driver. JDBC driver uses Thrift to communicate with the Hive Server.

3. ODBC client

Hive ODBC driver allows applications based on the ODBC protocol to connect to Hive. Similar to the JDBC driver, the ODBC driver uses Thrift to communicate with the Hive Server.

- Hive Service

To perform all queries, Hive provides various services like the Hive server2, Beeline, etc. The various services offered by Hive are:

1. Beeline

The Beeline is a command shell supported by HiveServer2, where the user can submit its queries and command to the system. It is a **JDBC** client that is based on **SQLLINE CLI** (pure Java-console-based utility for connecting with relational databases and executing SQL queries).

2. Hive Server 2

HiveServer2 is the successor of HiveServer1. HiveServer2 enables clients to execute queries against the Hive. It allows multiple clients to submit requests

to Hive and retrieve the final results. It is basically designed to provide the best support for open API clients like JDBC and ODBC.

Hive server1 does not handle concurrent requests from more than one client due to which it was replaced by HiveServer2.

3. Hive Driver

The Hive driver receives the **HiveQL** statements submitted by the user through the command shell. It creates the session handles for the query and sends the query to the compiler.

4. Hive Compiler

Hive compiler parses the query. It performs semantic analysis and type-checking on the different query blocks and query expressions by using the metadata stored in metastore and generates an execution plan.The execution plan created by the compiler is the **DAG(Directed Acyclic Graph)**, where each stage is a map/reduce job, operation on HDFS, a metadata operation.

5. Optimizer

Optimizer performs the transformation operations on the execution plan and splits the task to improve efficiency and scalability.

6. Execution Engine

Execution engine, after the compilation and optimization steps, executes the execution plan created by the compiler in order of their dependencies using Hadoop.

7. Metastore

Metastore is a central repository that stores the metadata information about the structure of tables and partitions, including column and column type information.

It also stores information of serializer and deserializer, required for the read/write operation, and HDFS files where data is stored. This metastore is generally a relational database.

Metastore provides a Thrift interface for querying and manipulating Hive metadata.

We can configure metastore in any of the two modes:

- **Remote:** In remote mode, metastore is a Thrift service and is useful for non-Java applications.
- **Embedded:** In embedded mode, the client can directly interact with the metastore using JDBC.

Processing Framework and Resource Management

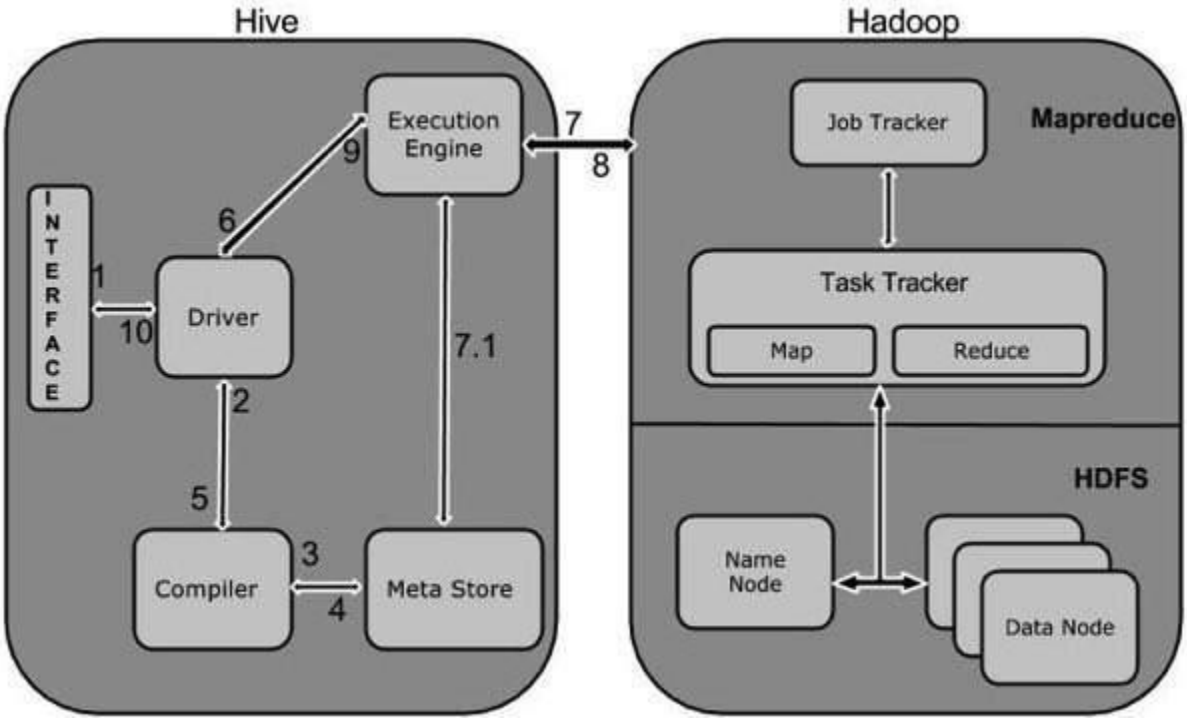
Hive internally uses a **MapReduce** framework as a defacto engine for executing the queries.
MapReduce is a software framework for writing those applications that process a massive amount of data in parallel on the large clusters of commodity hardware. MapReduce job works by splitting data into chunks, which are processed by map-reduce tasks.

Distributed Storage

Hive is built on top of Hadoop, so it uses the underlying Hadoop Distributed File System for the distributed storage.

How Does Hive Works?

The following diagram depicts the workflow between Hive and Hadoop.



Working of Hive

The following table defines how Hive interacts with Hadoop framework.

Step-1 Execute Query
At very first, the Hive interface (Command Line or Web UI) sends the query to Driver (any database driver such as JDBC, ODBC, etc.) to execute.

Step-2 Get Plan
Afterwards, the driver takes the help of query compiler which parses the query to check the syntax and query plan or the requirement of the query.

Step-3 Get Metadata
Further, the compiler sends metadata request to Metastore (any database).

Step-4 Send Metadata
After that Metastore sends metadata as a response to the compiler.

Step-5 Send Plan
Then the compiler checks the requirement and resends the plan to the driver. However, the parsing and compiling of a query are complete, Up to here.

Step-6 Execute Plan
Further, the driver sends the execution plan to the execution engine.

Step-7 Execute Job
Then, the process of execution job is a MapReduce job, internally. Also, the execution engine sends the job to JobTracker, which is in name node and it assigns this job to TaskTracker, which is in data node. Moreover, the query executes MapReduce job, here.

- **Metadata Ops**

During the execution, the execution engine can execute metadata operations with Metastore.

Step-8 Fetch Result
While execution is over, the execution engine receives the results from Data nodes.

Step-9 Send Results
After fetching results, execution engine sends those resultant values to the driver.

Step-10 Send Results
At last, the driver sends the results to Hive interfaces.

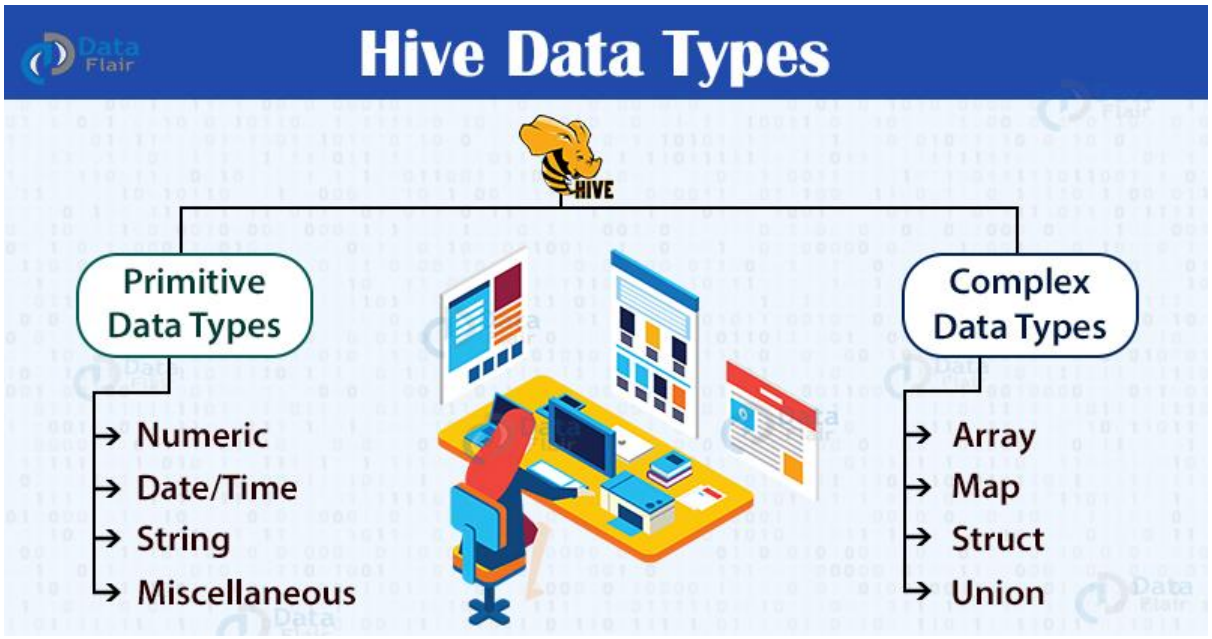
Examining the Hive Clients

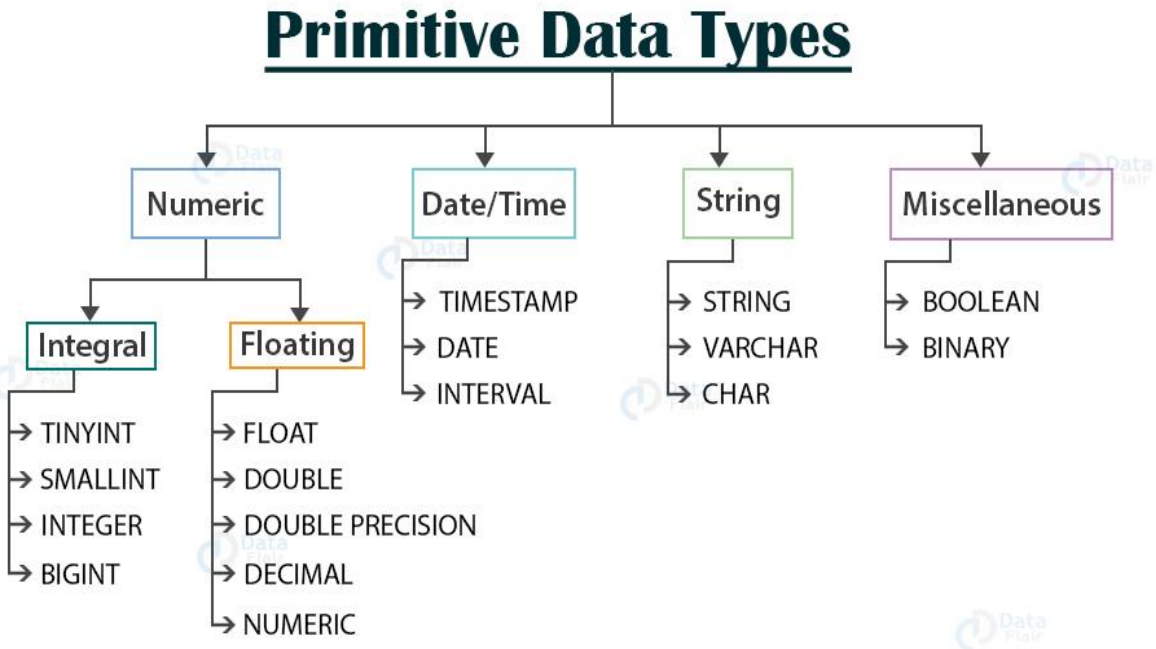
There are quite a number of client options for Hive as below.

1. Hive command-line interface (CLI)
2. Hive Web Interface (HWI) Server
3. Open source SQuirreL client using the JDBC driver.

Hive Data Types

Data Types in Hive specifies the column/field type in the Hive table. It specifies the type of values that can be inserted into the specified column.





Hive Complex Data Type

Complex Data Types are built on the top of Primitive Data Type.

The Hive Complex Data Type are categorized as:

Hive - Create Database

In Hive, the database is considered as a catalog or namespace of tables. So, we can maintain multiple tables within a database where a unique name is assigned to each table. Hive also provides a default database with a name **default**.

```
hive> show databases;
```

Let's create a new database by using the following command: -

```
hive> create database demo;
```

Let's check the existence of a newly created database.

```
hive> show databases;
```

If we want to suppress the warning generated by Hive on creating the database with the same name, follow the below command: -

```
hive> create a database if not exists demo;
```

Hive also allows assigning properties with the database in the form of key-value pair.

```
hive>create the database demo
    >WITH DBPROPERTIES ('creator' = 'Gaurav Chawla', 'date' = '2019-06-03');
```

Let's retrieve the information associated with the database.

```
hive> describe database extended demo;
```

```
hive> describe database extended demo;
OK
demo            hdfs://192.168.56.123:8020/user/hive/warehouse/demo.db  codegyan
i       USER    {date=2019-06-03, creator=Gaurav Chawla}
Time taken: 0.228 seconds, Fetched: 1 row(s)
hive>
```

Hive - Drop Database

- Let's check the list of existing databases by using the following command: -

hive> show databases;

Now, drop the database by using the following command.

hive> drop database demo

hive> drop database if exists demo;

```
hive> drop database if exists demo;
OK
Time taken: 0.056 seconds
hive>
```

- In Hive, it is not allowed to drop the database that contains the tables directly. In such a case, we can drop the database either by dropping tables first or use Cascade keyword with the command.
- Let's see the cascade command used to drop the database:-

  hive> drop database if exists demo cascade;

This command automatically drops the tables present in the database first.

Hive - Create Table

It provides two types of table: -

- Internal table
- External table

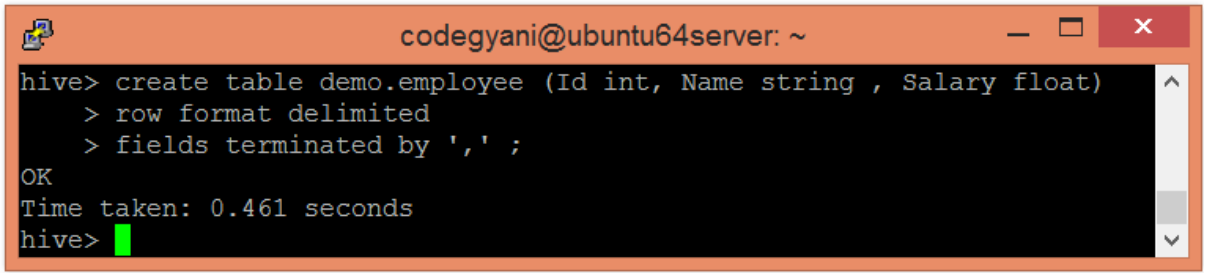
Internal Table

The internal tables are also called managed tables as the lifecycle of their data is controlled by the Hive.

By default, these tables are stored in a subdirectory under the directory defined by hive.metastore.warehouse.dir (i.e. /user/hive/warehouse).

If we try to drop the internal table, Hive deletes both table schema and data.

```
hive> create table demo.employee (Id int, Name string , Salary float)
        row format delimited  fields terminated by ',' ;
```



Here, the command also includes the information that the data is separated by ','.

- Let's see the metadata of the created table by using the following command:-

```
hive> describe demo.employee
```

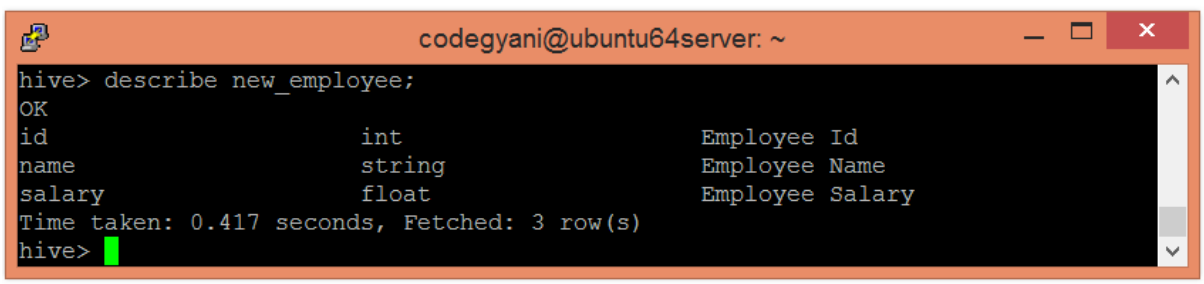
If we want to ignore this type of exception, we can use **if not exists** command while creating the table.

```
hive> create table if not exists demo.employee (Id int, Name string , Salary float)
        row format delimited fields terminated by ',' ;
```

While creating a table, we can add the comments to the columns and can also define the table properties.

```
hive> create table demo.new_employee (Id int comment 'Employee Id', Name string
      comment 'Employee Name', Salary float comment 'Employee Salary')
      comment 'Table Description'
     TBLProperties ('creator'='Gaurav Chawla', 'created_at' = '2019-06-06 11:00:00');
```

```
hive> describe new_employee;
```



- Hive allows creating a new table by using the schema of an existing table.

```
hive> create table if not exists demo.copy_employee  like demo.employee;
```

External Table

The external table allows us to create and access a table and a data externally. The **external** keyword is used to specify the external table, whereas the **location** keyword is used to determine the location of loaded data.

As the table is external, the data is not present in the Hive directory. Therefore, if we try to drop the table, the metadata of the table will be deleted, but the data still exists.

To create an external table, follow the below steps: -

- Let's create a directory on HDFS by using the following command: -

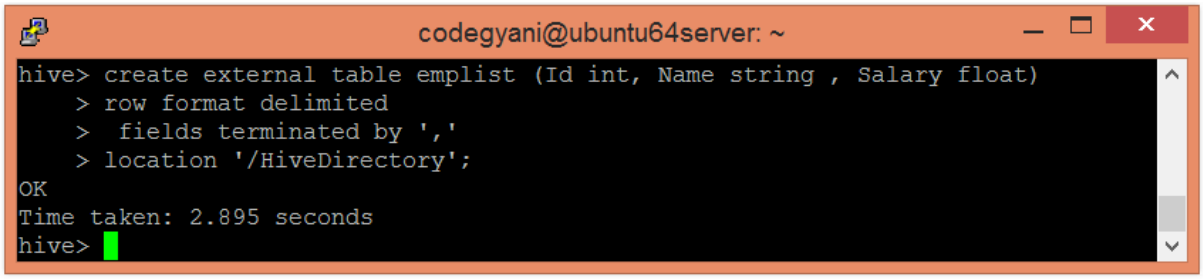
```
hdfs dfs -mkdir /HiveDirectory
```

- Now, store the file on the created directory.

```
hdfs dfs -put hive/emp_details/HiveDirectory
```

- Let's create an external table using the following command: -

```
hive> create external table emplist (Id int, Name string , Salary float)
        row format delimited
        fields terminated by ','
        location '/HiveDirectory';
```



- Now, we can use the following command to retrieve the data: -

select * from emplist;

```
codegyani@ubuntu64server: ~
hive> select * from emplist;
OK
1       "Gaurav"        30000.0
2       "Aryan" 20000.0
3       "Vishal"        40000.0
Time taken: 7.096 seconds, Fetched: 3 row(s)
hive>
```

Hive - Load Data

Once the internal table has been created, the next step is to load the data into it. So, in Hive, we can easily load data from any file to the database.

- Let's load the data of the file into the database by using the following command: -

load data local inpath '/home/codegyani/hive/emp_details'
into table demo.employee;

```
codegyani@ubuntu64server: ~/hive
hive> load data local inpath '/home/codegyani/hive/emp_details' into table demo.
employee;
Loading data to table demo.employee
Table demo.employee stats: [numFiles=1, numRows=0, totalSize=50, rawDataSize=0]
OK
Time taken: 7.505 seconds
hive>
```

Here, **emp_details** is the file name that contains the data.

- Now, we can use the following command to retrieve the data from the database.

- select * from demo.employee;

```
codegyani@ubuntu64server: ~/hive
hive> select * from demo.employee;
OK
1       "Gaurav"        30000.0
2       "Aryan" 20000.0
3       "Vishal"        40000.0
Time taken: 3.32 seconds, Fetched: 3 row(s)
hive>
```

Hive - Drop Table

Hive facilitates us to drop a table by using the SQL **drop table** command. Let's follow the below steps to drop the table from the database.

- Let's check the list of existing databases by using the following command: -

hive> show databases;

```
codegyani@ubuntu64server: ~
hive> show databases;
OK
default
demo
Time taken: 6.562 seconds, Fetched: 2 row(s)
hive>
```

- Now select the database from which we want to delete the table by using the following command: -

hive> use demo;

```
codegyani@ubuntu64server: ~
hive> use demo;
OK
Time taken: 0.148 seconds
hive>
```

- Let's check the list of existing tables in the corresponding database.

hive> show tables;

```
codegyani@ubuntu64server: ~
hive> show tables;
OK
emp
employee
new_employee
Time taken: 0.085 seconds, Fetched: 3 row(s)
hive>
```

- Now, drop the table by using the following command: -

hive> drop table new_employee;

```
codegyani@ubuntu64server: ~
hive> drop table new_employee;
OK
Time taken: 17.3 seconds
hive>
```

- Let's check whether the table is dropped or not.

hive> show tables;

```
codegyani@ubuntu64server: ~
hive> show tables;
OK
emp
employee
Time taken: 0.225 seconds, Fetched: 2 row(s)
hive>
```

As we can see, the table **new_employee** is not present in the list. Hence, the table is dropped successfully.

Hive - Alter Table

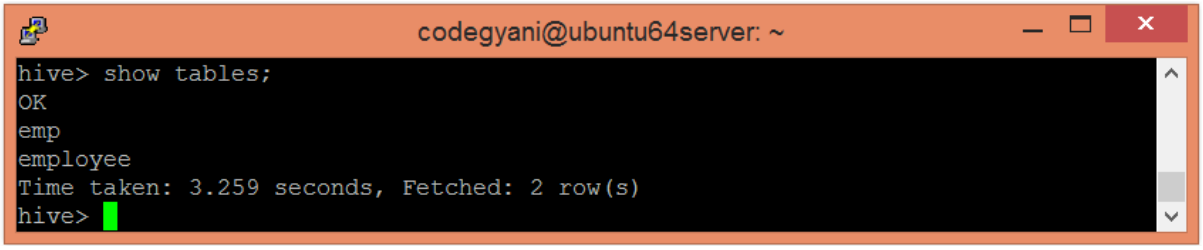
In Hive, we can perform modifications in the existing table like changing the table name, column name, comments, and table properties. It provides SQL like commands to alter the table.

Rename a Table

If we want to change the name of an existing table, we can rename that table by using the following signature: -

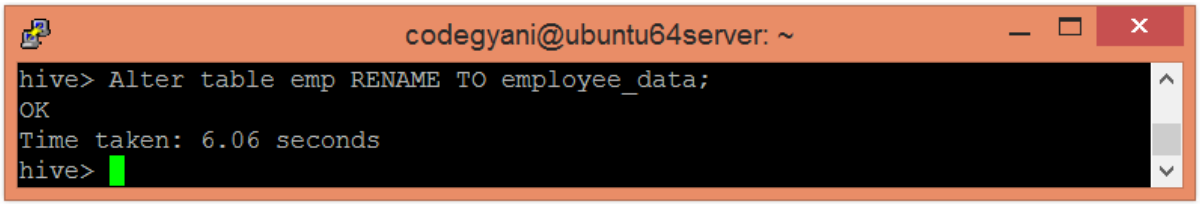
Alter table old_table_name rename to new_table_name;

- Let's see the existing tables present in the current database.



- Now, change the name of the table by using the following command: -

Alter table emp rename to employee_data;



Adding column

In Hive, we can add one or more columns in an existing table by using the following signature.

Alter table table_name add columns (column_name datatype);

- Let's see the schema of the table.

```
codegyani@ubuntu64server: ~
hive> describe employee_data;
OK
id                      int
name                    string
salary                  float
Time taken: 0.564 seconds, Fetched: 3 row(s)
hive>
```

- Let's see the data of columns exists in the table.

```
codegyani@ubuntu64server: ~
hive> select * from employee_data;
OK
1       "Gaurav"        30000.0
2       "Aryan" 20000.0
3       "Vishal"        40000.0
Time taken: 0.555 seconds, Fetched: 3 row(s)
hive>
```

- Now, add a new column to the table by using the following command: -

Alter table employee_data add columns (age int);

```
codegyani@ubuntu64server: ~
hive> Alter table employee_data add columns ( age int);
OK
Time taken: 1.025 seconds
hive>
```

Change Column

In Hive, we can rename a column; change its type and position. Here, we are changing the name of the column by using the following signature: -

Alter table table_name change *old_column_name new_column_name* datatype;

- Let's see the existing schema of the table.

```
codegyani@ubuntu64server: ~
hive> describe employee_data;
OK
id                      int
name                    string
salary                  float
age                     int
Time taken: 0.308 seconds, Fetched: 4 row(s)
hive>
```

- Now, change the name of the column by using the following command: -

Alter table employee_data change name first_name string;

```
codegyani@ubuntu64server: ~
hive> Alter table employee_data change name first_name string;
OK
Time taken: 0.835 seconds
hive>
```

- Let's check whether the column name has changed or not.

```
codegyani@ubuntu64server: ~
hive> describe employee_data;
OK
id                      int
first_name              string
salary                  float
age                     int
Time taken: 0.372 seconds, Fetched: 4 row(s)
hive>
```

Delete or Replace Column

Hive allows us to delete one or more columns by replacing them with the new columns. Thus, **we cannot drop the column directly.**

- Let's see the existing schema of the table.

```
codegyani@ubuntu64server: ~
hive> describe employee_data;
OK
id                      int
name                    string
salary                  float
age                     int
Time taken: 0.308 seconds, Fetched: 4 row(s)
hive>
```

- Now, drop a column from the table.

alter table employee_data replace columns(id string, first_name string, age int);

```
codegyani@ubuntu64server: ~
hive> alter table employee_data replace columns( id string, first_name string, a
ge int);
OK
Time taken: 0.544 seconds
hive>
```

- Let's check whether the column has dropped or not.

```
codegyani@ubuntu64server: ~
hive> describe employee_data;
OK
id                      string
first_name              string
age                     int
Time taken: 0.337 seconds, Fetched: 3 row(s)
hive>
```

Hive Different File Formats

Following are the Apache Hive different file formats:

- Text File
- Sequence File
- RC File
- AVRO File
- ORC File
- Parquet File

<u>Hive Text File Format</u>

- **Hive Text file format** is a default storage format. You can use the text format to interchange the data with other client application. The text file format is very common most of the applications.
- Data is stored in lines, with each line being a record. Each lines are terminated by a newline character (\n).
- The text format is simple plane file format. You can use the compression (*BZIP2*) on the text file to reduce the storage spaces.

```
Create table textfile_table        (column_specs)        stored as textfile;
```

<u>Hive Sequence File Format</u>

- **Sequence files** are Hadoop flat files which stores values in binary key-value pairs. The sequence files are in binary format and these files are able to split.
- The main advantage of using sequence file is to merge two or more files into one file.

```
Create table sequencefile_table     (column_specs)      stored as sequencefile;
```

<u>Hive RC File Format</u>

- **RCFile** is row columnar file format. This is another form of Hive file format which offers high row level compression rates. If you have requirement to perform multiple rows at a time then you can use RCFile format.
- The RCFile are very much similar to the sequence file format. This file format also stores the data as key-value pairs.

```
Create table RCfile_table   (column_specs)    stored as rcfile;
```

Hive AVRO File Format

- **AVRO** is open source project that provides data serialization and data exchange services for Hadoop. You can exchange data between Hadoop ecosystem and program written in any programming languages.
- Avro is one of the popular file formats in Big Data Hadoop based applications.

```
Create table avro_table    (column_specs)    stored as avro;
```

Hive ORC File Format

- The **ORC file** stands for Optimized Row Columnar file format. The ORC file format provides a highly efficient way to store data in Hive table. This file system was actually designed to overcome limitations of the other Hive file formats.
- The Use of ORC files improves performance when Hive is reading, writing, and processing data from large tables.

```
Create table orc_table       (column_specs)       stored as orc;
```

Hive Parquet File Format

- **Parquet** is a column-oriented binary file format. The parquet is highly efficient for the types of large-scale queries.
- Parquet is especially good for queries scanning particular columns within a particular table. The Parquet table uses compression Snappy, gzip; currently Snappy by default.

```
Create table parquet_table    (column_specs)     stored as parquet;
```

Arithmetic Operators in Hive

In Hive, the arithmetic operator accepts any numeric type. The commonly used arithmetic operators are: -

Operators	Description
A + B	This is used to add A and B.
A - B	This is used to subtract B from A.
A * B	This is used to multiply A and B.
A / B	This is used to divide A and B and returns the quotient of the operands.
A % B	This returns the remainder of A / B.
A | B	This is used to determine the bitwise OR of A and B.
A & B	This is used to determine the bitwise AND of A and B.
A ^ B	This is used to determine the bitwise XOR of A and B.
~A	This is used to determine the bitwise NOT of A.

Examples of Arithmetic Operator in Hive

- Let's see an example to increase the salary of each employee by 50.

```
hive> select id, name, salary + 50 from employee;
```

```
codegyani@ubuntu64server: ~/hive
hive> select id, name, salary + 50 from employee;
OK
1       "Gaurav"        30050.0
2       "Aryan" 20050.0
3       "Vishal"        40050.0
4       "John"  10050.0
5       "Henry" 25050.0
6       "William"       9050.0
7       "Lisa"  25050.0
8       "Ronit" 20050.0
NULL    NULL    NULL
Time taken: 0.445 seconds, Fetched: 9 row(s)
hive>
```

- Let's see an example to decrease the salary of each employee by 50.

hive> select id, name, salary - 50 from employee;

```
codegyani@ubuntu64server: ~/hive
hive> select id, name, salary - 50 from employee;
OK
1       "Gaurav"        29950.0
2       "Aryan" 19950.0
3       "Vishal"        39950.0
4       "John"  9950.0
5       "Henry" 24950.0
6       "William"       8950.0
7       "Lisa"  24950.0
8       "Ronit" 19950.0
NULL    NULL    NULL
Time taken: 0.869 seconds, Fetched: 9 row(s)
hive>
```

- Let's see an example to find out the 10% salary of each employee.

hive> select id, name, (salary * 10) /100 from employee;

Relational Operators in Hive

In Hive, the relational operators are generally used with clauses like Join and Having to compare the existing records. The commonly used relational operators are: -

Operator	Description
A=B	It returns true if A equals B, otherwise false.
A <> B, A !=B	It returns null if A or B is null; true if A is not equal to B, otherwise false.
A<B	It returns null if A or B is null; true if A is less than B, otherwise false.
A>B	It returns null if A or B is null; true if A is greater than B, otherwise false.
A<=B	It returns null if A or B is null; true if A is less than or equal to B, otherwise false.
A>=B	It returns null if A or B is null; true if A is greater than or equal to B, otherwise false.
A IS NULL	It returns true if A evaluates to null, otherwise false.
A IS NOT NULL	It returns false if A evaluates to null, otherwise true.

Examples of Relational Operator in Hive

- Let's see an example to fetch the details of the employee having salary>=25000.

```
hive> select * from employee where salary >= 25000;
```

```
codegyani@ubuntu64server: ~/hive
hive> select * from employee where salary >= 25000;
OK
1       "Gaurav"        30000.0
3       "Vishal"        40000.0
5       "Henry" 25000.0
7       "Lisa"  25000.0
Time taken: 1.297 seconds, Fetched: 4 row(s)
hive>
```

- Let's see an example to fetch the details of the employee having salary<25000.

hive> select * from employee where salary < **25000**;

```
codegyani@ubuntu64server: ~/hive
hive> select * from employee where salary < 25000;
OK
2       "Aryan" 20000.0
4       "John"  10000.0
6       "William"       9000.0
8       "Ronit" 20000.0
Time taken: 0.727 seconds, Fetched: 4 row(s)
hive>
```

Mathematical Functions in Hive

The commonly used mathematical functions in the hive are: -

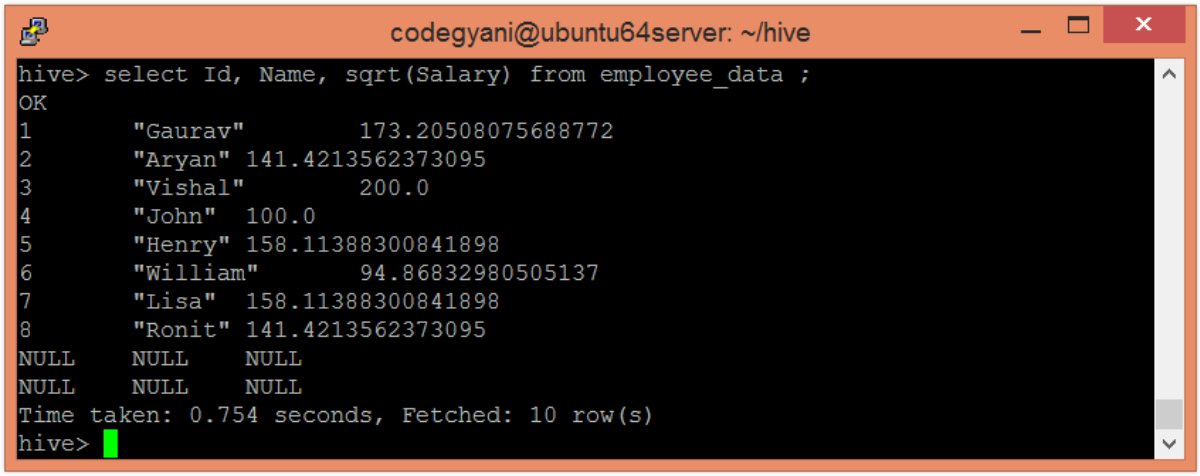
Return type	Functions	Description
BIGINT	round(num)	It returns the BIGINT for the rounded value of DOUBLE num.
BIGINT	floor(num)	It returns the largest BIGINT that is less than or equal to num.
BIGINT	ceil(num), ceiling(DOUBLE num)	It returns the smallest BIGINT that is greater than or equal to num.
DOUBLE	exp(num)	It returns exponential of num.
DOUBLE	ln(num)	It returns the natural logarithm of num.
DOUBLE	log10(num)	It returns the base-10 logarithm of num.
DOUBLE	sqrt(num)	It returns the square root of num.
DOUBLE	abs(num)	It returns the absolute value of num.
DOUBLE	sin(d)	It returns the sin of num, in radians.
DOUBLE	asin(d)	It returns the arcsin of num, in radians.
DOUBLE	cos(d)	It returns the cosine of num, in radians.
DOUBLE	acos(d)	It returns the arccosine of num, in radians.

DOUBLE	tan(d)	It returns the tangent of num, in radians.
DOUBLE	atan(d)	It returns the arctangent of num, in radians.

Example of Mathematical Functions in Hive

- Let's see an example to fetch the square root of each employee's salary.

hive> select Id, Name, sqrt(Salary) from employee_data ;



Aggregate Functions in Hive

In Hive, the aggregate function returns a single value resulting from computation over many rows. Let''s see some commonly used aggregate functions: -

Return Type	Operator	Description
BIGINT	count(*)	It returns the count of the number of rows present in the file.
DOUBLE	sum(col)	It returns the sum of values.
DOUBLE	sum(DISTINCT col)	It returns the sum of distinct values.
DOUBLE	avg(col)	It returns the average of values.
DOUBLE	avg(DISTINCT col)	It returns the average of distinct values.
DOUBLE	min(col)	It compares the values and returns the minimum one form it.
DOUBLE	max(col)	It compares the values and returns the maximum one form it.

Examples of Aggregate Functions in Hive

- Let's see an example to fetch the maximum salary of an employee.

hive> select max(Salary) from employee_data;

```
codegyani@ubuntu64server: ~/hive
Total MapReduce CPU Time Spent: 23 seconds 840 msec
OK
40000.0
Time taken: 231.973 seconds, Fetched: 1 row(s)
hive>
```

- Let's see an example to fetch the minimum salary of an employee.

hive> select min(Salary) from employee_data;

```
codegyani@ubuntu64server: ~/hive
Total MapReduce CPU Time Spent: 24 seconds 20 msec
OK
9000.0
Time taken: 275.472 seconds, Fetched: 1 row(s)
hive>
```

Other built-in Functions in Hive

The following are some other commonly used in-built functions in the hive: -

Return Type	Operator	Description
INT	length(str)	It returns the length of the string.
STRING	reverse(str)	It returns the string in reverse order.
STRING	concat(str1, str2, ...)	It returns the concatenation of two or more strings.
STRING	substr(str, start_index)	It returns the substring from the string based on the provided starting index.
STRING	substr(str, int start, int length)	It returns the substring from the string based on the provided starting index and length.
STRING	upper(str)	It returns the string in uppercase.
STRING	lower(str)	It returns the string in lowercase.
STRING	trim(str)	It returns the string by removing whitespaces from both the ends.
STRING	ltrim(str)	It returns the string by removing whitespaces from left-hand side.
TRING	rtrim(str)	It returns the string by removing whitespaces from right-hand side.

Examples of other in-built Functions in Hive

- Let's see an example to fetch the name of each employee in uppercase.

select Id, upper(Name) from employee_data;

```
codegyani@ubuntu64server: ~/hive
hive> select Id, upper(Name) from employee_data;
OK
1       "GAURAV"
2       "ARYAN"
3       "VISHAL"
4       "JOHN"
5       "HENRY"
6       "WILLIAM"
7       "LISA"
8       "RONIT"
NULL    NULL
NULL    NULL
Time taken: 0.529 seconds, Fetched: 10 row(s)
hive>
```

- Let's see an example to fetch the name of each employee in lowercase.

select Id, lower(Name) from employee_data;

```
codegyani@ubuntu64server: ~/hive
hive> select Id, lower(Name) from employee_data;
OK
1       "gaurav"
2       "aryan"
3       "vishal"
4       "john"
5       "henry"
6       "william"
7       "lisa"
8       "ronit"
NULL    NULL
NULL    NULL
Time taken: 0.516 seconds, Fetched: 10 row(s)
hive>
```

HiveQL - GROUP BY and HAVING Clause

emp

Id	Name	Salary	Department
1	Gaurav	30000	Developer
2	Aryan	20000	Manager
3	Vishal	40000	Manager
4	John	10000	Trainer
5	Henry	25000	Developer
6	William	9000	Developer
7	Lisa	25000	Manager
8	Ronit	20000	Trainer

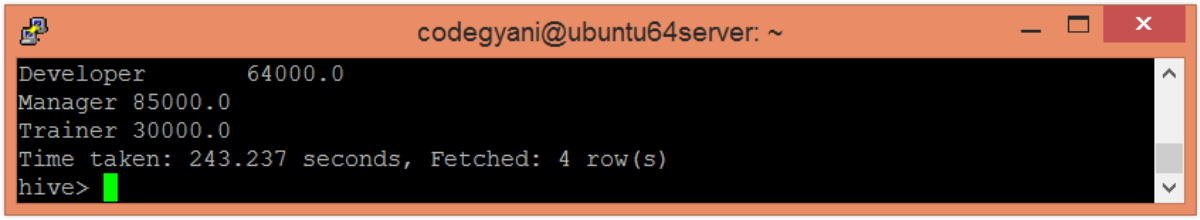
GROUP BY Clause

The **HQL Group By** clause is used to group the data from the multiple records based on one or more column.

It is generally used in conjunction with the aggregate functions (like SUM, COUNT, MIN, MAX and AVG) to perform an aggregation over each group.

Example of GROUP BY Clause in Hive

```
hive> select department, sum(salary) from emp group by department;
```



HAVING CLAUSE

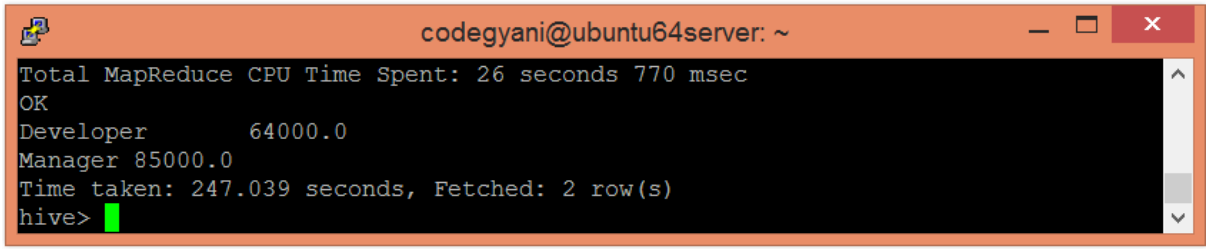
The HQL **HAVING clause** is used with **GROUP BY** clause. Its purpose is to apply constraints on the group of data produced by GROUP BY clause. Thus, it always returns the data where the condition is **TRUE**.

Example of Having Clause in Hive

In this example, we fetch the sum of employee's salary based on department and apply the required constraints on that sum by using HAVING clause.

- Let's fetch the sum of employee's salary based on department having sum >= 35000 by using the following command:

```
hive> select department, sum(salary) from emp group by department
                                having sum(salary)>=35000;
```



Here, we got the desired output.

HiveQL - ORDER BY and SORT BY Clause

By using HiveQL ORDER BY and SORT BY clause, we can apply sort on the column.

It returns the result set either in ascending or descending order.

Here, we are going to execute these clauses on the records of the below table:

emp

Id	Name	Salary	Department
1	Gaurav	30000	Developer
2	Aryan	20000	Manager
3	Vishal	40000	Manager
4	John	10000	Trainer
5	Henry	25000	Developer
6	William	9000	Developer
7	Lisa	25000	Manager
8	Ronit	20000	Trainer

HiveQL - ORDER BY Clause

In HiveQL, ORDER BY clause performs a complete ordering of the query result set. Hence, the complete data is passed through a single reducer. This may take much time in the execution of large datasets. However, we can use LIMIT to minimize the sorting time.

Example of ORDER BY Clause in Hive

```
hive> select * from emp order by salary desc;
```

```
codegyani@ubuntu64server: ~
Total MapReduce CPU Time Spent: 22 seconds 340 msec
OK
3       "Vishal"        40000.0 Manager
1       "Gaurav"        30000.0 Developer
7       "Lisa"  25000.0 Manager
5       "Henry" 25000.0 Developer
8       "Ronit" 20000.0 Trainer
2       "Aryan" 20000.0 Manager
4       "John"  10000.0 Trainer
6       "William"       9000.0  Developer
NULL    NULL    NULL    NULL
Time taken: 257.304 seconds, Fetched: 9 row(s)
hive>
```

HiveQL - SORT BY Clause

The HiveQL SORT BY clause is an alternative of ORDER BY clause. It orders the data within each reducer. Hence, it performs the local ordering, where each reducer's output is sorted separately. It may also give a partially ordered result.

Example of SORT BY Clause in Hive

In this example, we arrange the data in the sorted order by using SORT BY clause.

- Let's fetch the data in the descending order by using the following command:

hive> select * from emp sort by salary desc;

```
codegyani@ubuntu64server: ~
Total MapReduce CPU Time Spent: 22 seconds 690 msec
OK
3       "Vishal"        40000.0 Manager
1       "Gaurav"        30000.0 Developer
7       "Lisa"  25000.0 Manager
5       "Henry" 25000.0 Developer
8       "Ronit" 20000.0 Trainer
2       "Aryan" 20000.0 Manager
4       "John"  10000.0 Trainer
6       "William"       9000.0  Developer
NULL    NULL    NULL    NULL
Time taken: 268.62 seconds, Fetched: 9 row(s)
```

HiveQL - JOIN

The HiveQL Join clause is used to combine the data of two or more tables based on a related column between them. The various type of HiveQL joins are: -

- Inner Join
- Left Outer Join
- Right Outer Join
- Full Outer Join

Here, we are going to execute the join clauses on the records of the following table:

employee

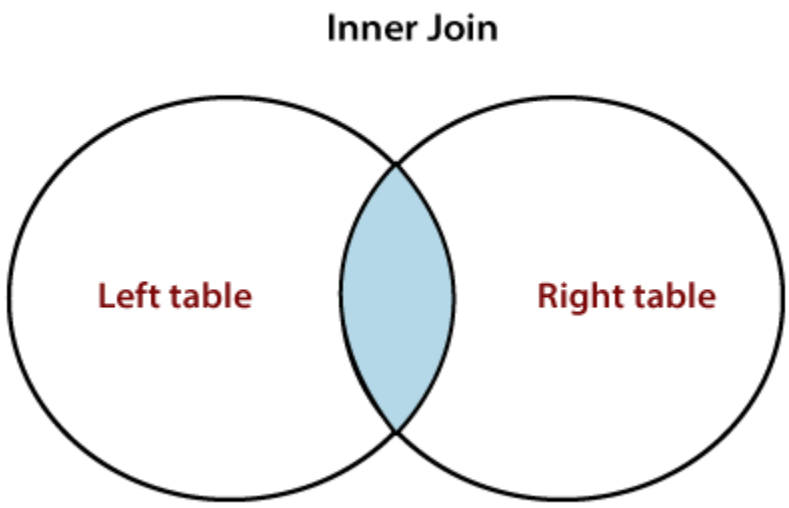
empid	empname	state
1	Gaurav	UP
2	Aryan	Punjab
3	Vishal	UP
4	John	Haryana
5	Henry	UP

employee_department

depid	department_name
2	IT
2	Trainer
3	Manager
4	Admin

Inner Join in HiveQL

The HiveQL inner join is used to return the rows of multiple tables where the join condition satisfies. In other words, the join criteria find the match records in every table being joined.

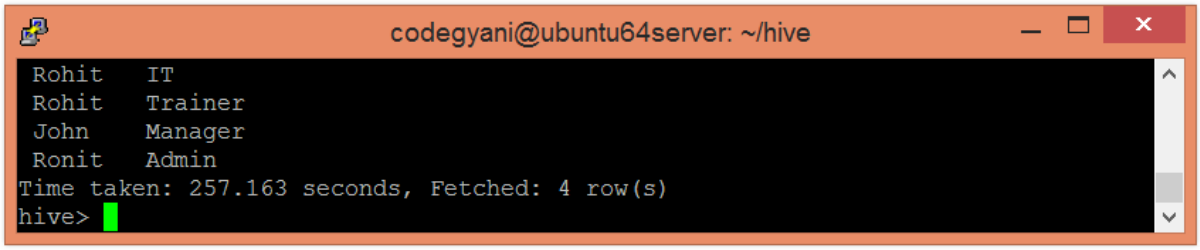


Example of Inner Join in Hive

In this example, we take two table employee and employee_department. The primary key (empid) of employee table represents the foreign key (depid) of employee_department table.

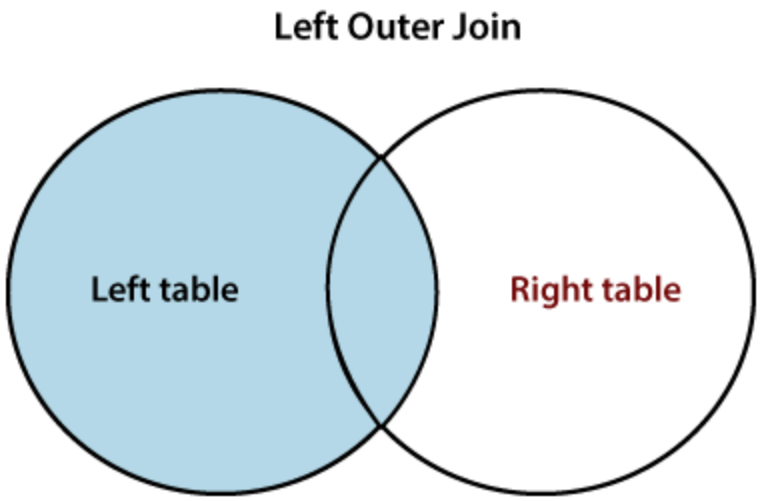
Now, perform the inner join operation by using the following command: -

```
hive>select  e1.empname, e2.department_name
        from employee e1 join employee_department e2 on e1.empid= e2.depid;
```



Left Outer Join in HiveQL

The HiveQL left outer join returns all the records from the left (first) table and only that records from the right (second) table where join criteria find the match.



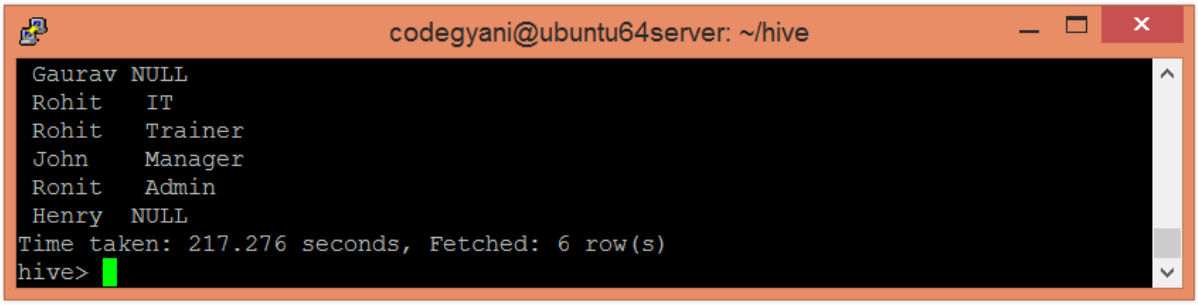
Example of Left Outer Join in Hive

In this example, we perform the left outer join operation.

- Let's us execute the left outer join operation by using the following command: -

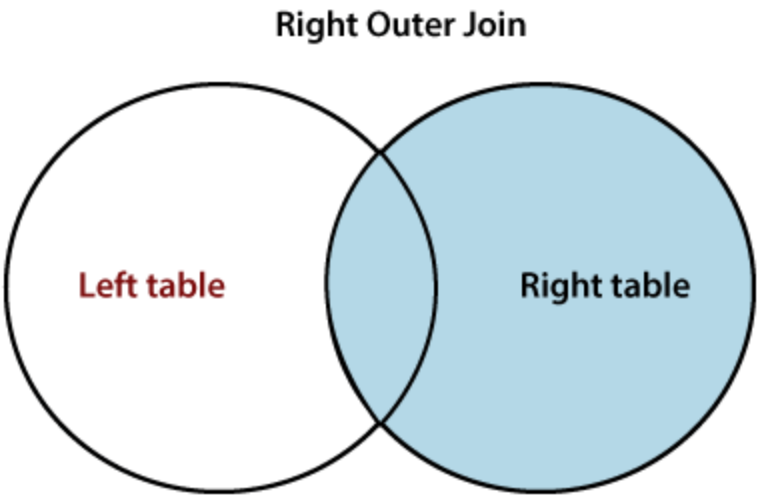
hive> select e1.empname, e2.department_name

from employee e1 left outer join employee_department e2 on e1.empid= e2.depid;



Right Outer Join in HiveQL

The HiveQL right outer join returns all the records from the right (second) table and only that records from the left (first) table where join criteria find the match.



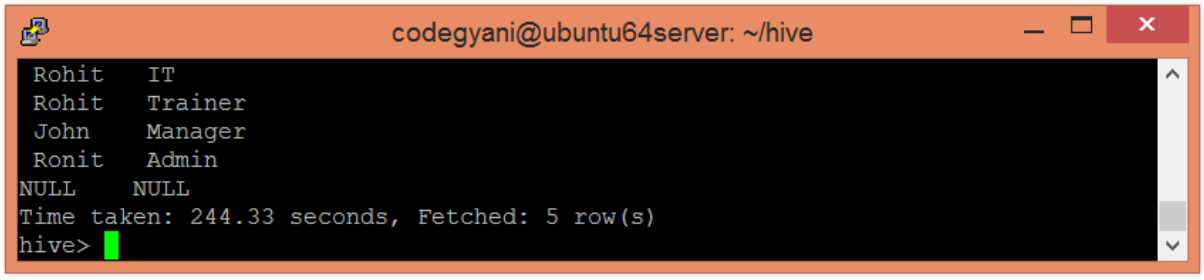
Example of Left Outer Join in Hive

In this example, we perform the left outer join operation.

- Let's us execute the left outer join operation by using the following command: -

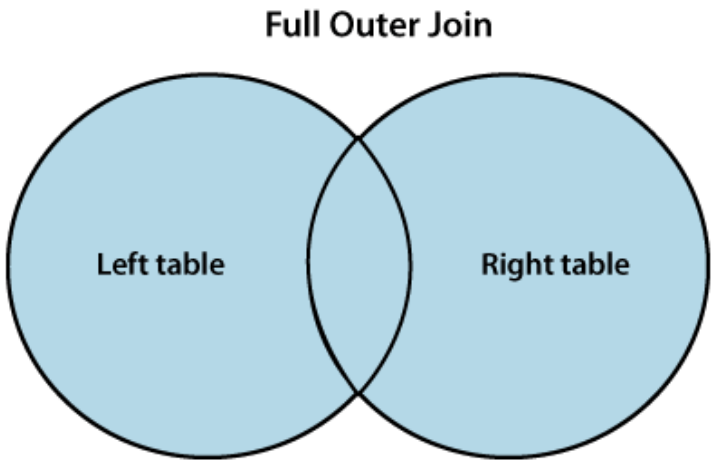
hive> select e1.empname, e2.department_name

from employee e1 right outer join employee_department e2 on e1.empid= e2.depid;



Full Outer Join

The HiveQL full outer join returns all the records from both the tables. It assigns Null for missing records in either table.



Example of Full Outer Join in Hive

In this example, we perform the full outer join operation.

- Let's us execute the full outer join operation by using the following command: -

```
select  e1.empname, e2.department_name
from employee e1 full outer join employee_department e2 on e1.empid= e2.depid;
```

Types of Hive Partitioning

Till now we have discussed Introduction to Hive Partitions and How to create Hive partitions. Now we are going to introduce the types of data partitioning in Hive. There are two types of Partitioning in Apache Hive-

- Static Partitioning
- Dynamic Partitioning

Let's discuss these types of Hive Partitioning one by one-

i. Hive Static Partitioning

- Insert input data files individually into a partition table is Static Partition.
- Usually when loading files (big files) into **Hive tables** static partitions are preferred.
- Static Partition saves your time in loading data compared to dynamic partition.
- You "statically" add a partition in the table and move the file into the partition of the table.
- We can alter the partition in the static partition.
- You can get the partition column value from the filename, day of date etc without reading the whole big file.
- If you want to use the Static partition in the hive you should set property **set hive.mapred.mode = strict** This property set by default in hive-site.xml
- Static partition is in Strict Mode.
- You should use where clause to use limit in the static partition.
- You can perform Static partition on Hive Manage table or external table.

```
hive>LOAD DATA INPATH '/hdfs path of the file' INTO TABLE t1 PARTITION(country="US")
hive>LOAD DATA INPATH '/hdfs path of the file' INTO TABLE t1 PARTITION(country="UK")
```

ii. Hive Dynamic Partitioning

- Single insert to partition table is known as a dynamic partition.
- Usually, dynamic partition loads the data from the non-partitioned table.
- Dynamic Partition takes more time in loading data compared to static partition.

- When you have large data stored in a table then the Dynamic partition is suitable.
- If you want to partition a number of columns but you don't know how many columns then also dynamic partition is suitable.
- Dynamic partition there is no required where clause to use limit.
- we can't perform alter on the Dynamic partition.
- You can perform dynamic partition on hive external table and managed table.
- If you want to use the Dynamic partition in the hive then the mode is in non-strict mode.

```
hive> INSERT INTO TABLE t2 PARTITION(country) SELECT * from T1;
```

Hive Partitioning – Advantages and Disadvantages

Let's discuss some benefits and limitations of Apache Hive Partitioning-

a) Hive Partitioning Advantages

- Partitioning in Hive distributes execution load horizontally.
- In partition faster execution of queries with the low volume of data takes place. For example, search population from Vatican City returns very fast instead of searching entire world population.

b) Hive Partitioning Disadvantages

- There is the possibility of too many small partition creations- too many directories.
- Partition is effective for low volume data. But there some queries like group by on high volume of data take a long time to execute. For example, grouping population of China will take a long time as compared to a grouping of the population in Vatican City.
- There is no need for searching entire table column for a single record.

Bucketing in Hive – Creation of Bucketed Table in Hive

In **Apache Hive**, for decomposing table data sets into more manageable parts, it uses Hive Bucketing concept.

What is Bucketing in Hive

Basically, for decomposing table data sets into more manageable parts, Apache Hive offers another technique. That technique is what we call Bucketing in Hive.

Why Bucketing?

Basically, the concept of **Hive Partitioning** provides a way of segregating hive table data into multiple files/directories. However, it only gives effective results in few scenarios. Such as:

– When there is the limited number of partitions.
– Or, while partitions are of comparatively equal size.

Although, it is not possible in all scenarios. For example when are partitioning our tables based geographic locations like country. Hence, some bigger countries will have large partitions (ex: 4-5 countries itself contributing 70-80% of total data).
While small countries data will create small partitions (remaining all countries in the world may contribute to just 20-30 % of total data). Hence, at that time Partitioning will not be ideal.

Then, to solve that problem of over partitioning, Hive offers Bucketing concept. It is another effective technique for decomposing table data sets into more manageable parts.

Features of Bucketing in Hive

Basically, this concept is based on hashing function on the bucketed column. Along with mod (by the total number of buckets).
i. Where the hash_function depends on the type of the bucketing column.
ii. However, the Records with the same bucketed column will always be stored in the

same bucket.
iii. Moreover, to divide the table into buckets we use CLUSTERED BY clause.
iv. Generally, in the table directory, each bucket is just a file, and Bucket numbering is 1-based.
v. Along with Partitioning on Hive tables bucketing can be done and even without partitioning.
vi. Moreover, Bucketed tables will create almost equally distributed data file parts.

Advantages of Bucketing in Hive

i. On comparing with non-bucketed tables, Bucketed tables offer the efficient sampling.
ii. Map-side joins will be faster on bucketed tables than non-bucketed tables, as the data files are equal sized parts.
iii. Here also bucketed tables offer faster query responses than non-bucketed tables as compared to Similar to partitioning.
iv. This concept offers the flexibility to keep the records in each bucket to be sorted by one or more columns.
v. Since the join of each bucket becomes an efficient merge-sort, this makes map-side joins even more efficient.

Limitations of Bucketing in Hive

i. However, it doesn't ensure that the table is properly populated.
ii. So, we need to handle Data Loading into buckets by our-self.

Example Use Case for Bucketing in Hive

To understand the remaining features of Hive Bucketing let's see an example Use case, by creating buckets for the sample user records file for testing in this post

first_name,last_name, address, country, city, state, post,phone1,phone2, email, web Rebbecca, Didio, 171 E 24th St, AU, Leith, TA, 7315, 03-8174-9123, 0458-665-290, rebbecca.didio@didio.com.au,http://www.brandtjonathanfesq.com.au

Hence, let's create the table partitioned by country and bucketed by state and sorted in ascending order of cities.

Creation of Bucketed Tables

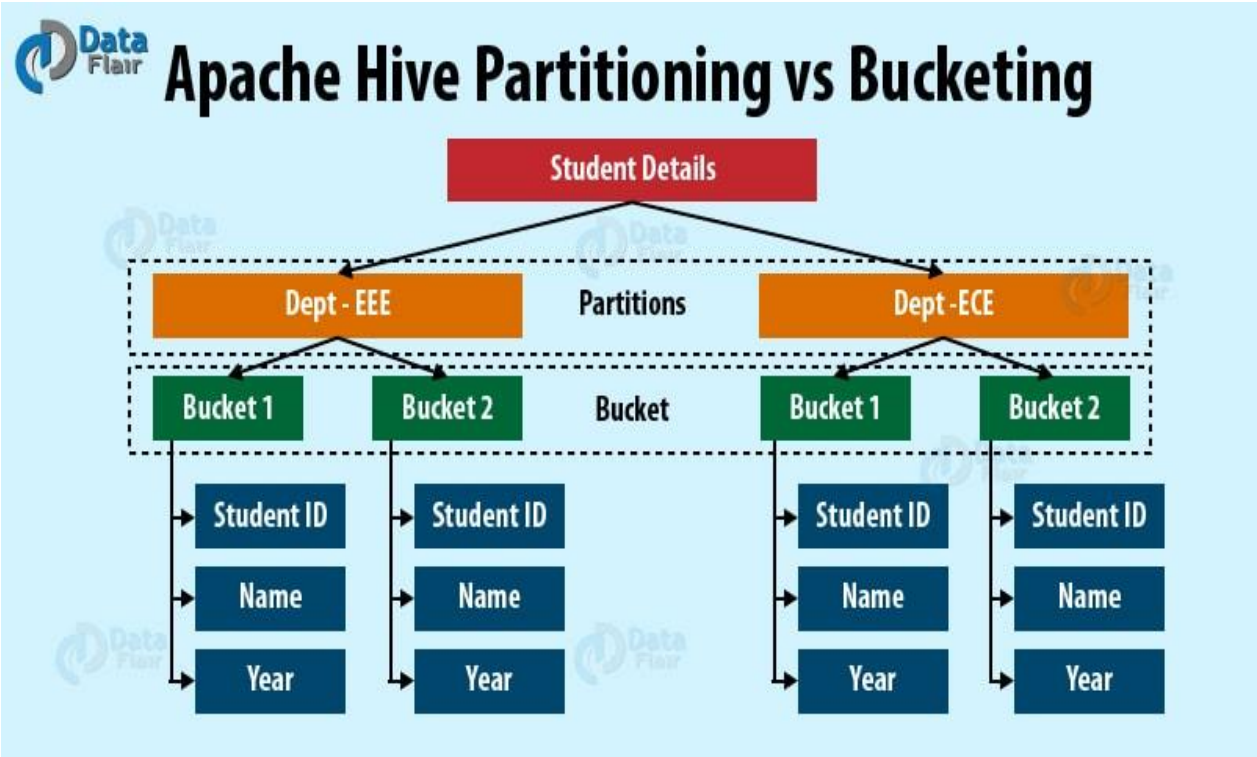
However, with the help of CLUSTERED BY clause and optional SORTED BY clause in CREATE TABLE statement we can create bucketed tables. Moreover, we can create a bucketed_user table with above-given requirement with the help of the below HiveQL.

```
CREATE TABLE bucketed_user(
      firstname VARCHAR(64),
       lastname  VARCHAR(64),
       address   STRING,
       city  VARCHAR(64),
      state  VARCHAR(64),
       post      STRING,
       phone1    VARCHAR(64),
       phone2    STRING,
       email     STRING,
       web       STRING
       )
      COMMENT ‘A bucketed sorted user table’
       PARTITIONED BY (country VARCHAR(64))
      CLUSTERED BY (state) SORTED BY (city) INTO 32 BUCKETS
       STORED AS SEQUENCEFILE;
```

What is Hive Partitioning and Bucketing?

Apache Hive is an open source data warehouse system used for querying and analyzing large datasets.
Data in Apache Hive can be categorized into **Table, Partition, and Bucket**.

- **Partitioning –** Apache Hive organizes tables into partitions for grouping same type of data together based on a column or partition key. Each table in the hive can have one or more partition keys to identify a particular partition. Using partition we can make it faster to do queries on slices of the data.
- **Bucketing –** In Hive Tables or partition are subdivided into buckets based on the hash function of a column in the table to give extra structure to the data that may be used for more efficient queries.



Partitioning and Bucketing Commands in Hive

a) Hive Partitioning Example

- For example, we have a table employee_details containing the employee information of some company like ***employee_id, name, department, year,*** etc.
- Now, if we want to perform partitioning on the basis of department column. Then the information of all the employees belonging to a particular department will be stored together in that very partition.
- Physically, a partition in Hive is nothing but just a sub-directory in the table directory.

For example, we have data for three departments in our employee_details table – Technical, Marketing and Sales. Thus we will have three partitions in total for each of the departments.

So for example, all the employee data regarding Technical departments will be stored in user/hive/warehouse/employee_details/dept.=Technical.

So, the queries regarding Technical employee would only have to look through the data present in the Technical partition.

Therefore from above example, we can conclude that partitioning is very useful. It reduces the query latency by scanning only **relevant** partitioned data instead of the whole data set.

The Hive command for Partitioning is:

```
CREATE TABLE table_name (column1 data_type, column2 data_type)
PARTITIONED BY (partition1 data_type, partition2 data_type,….);
```

a) Pros and Cons of Hive Partitioning

Pros:

- It distributes execution load horizontally.
- In partition faster execution of queries with the low volume of data takes place.

Cons:

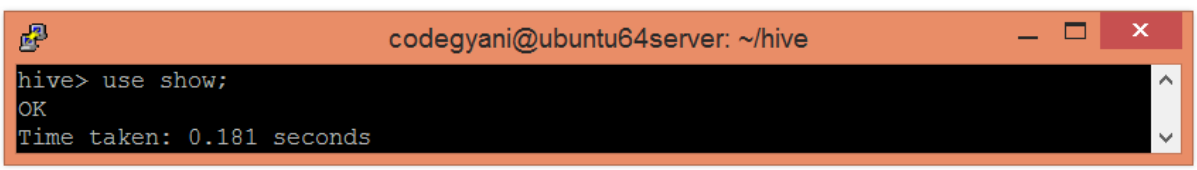
- There is the possibility of too many small partition creations- too many directories.
- Partition is effective for low volume data. But there some queries like group by on high volume of data take a long time to execute.

Dynamic Partitioning

In dynamic partitioning, the values of partitioned columns exist within the table. So, it is not required to pass the values of partitioned columns manually.

- First, select the database in which we want to create a table.

```
hive> use show;
```



- Enable the dynamic partition by using the following commands: -

```
hive> set hive.exec.dynamic.partition=true;
hive> set hive.exec.dynamic.partition.mode=nonstrict;
```

- Create a dummy table to store the data.

```
hive> create table stud_demo(id int, name string, age int, institute string,
                                course string)
      row format delimited  fields terminated by ',';
```

Now, load the data into the table.

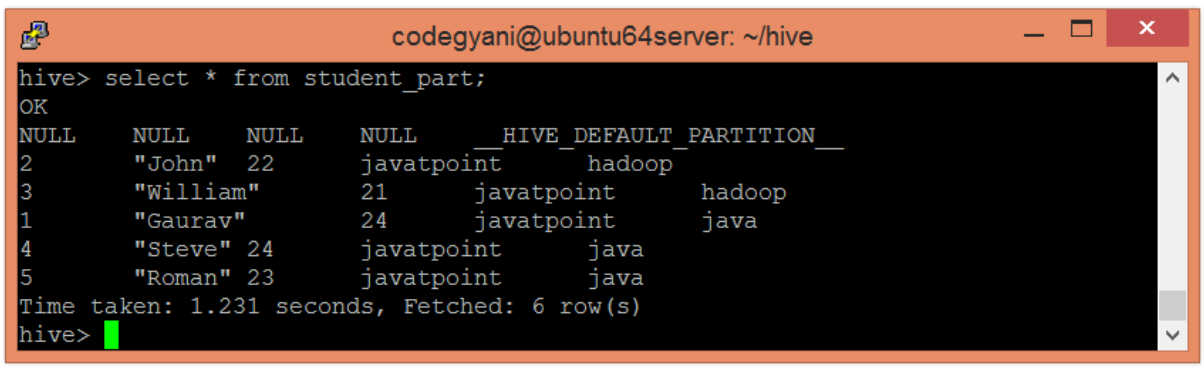
```
hive> load data local inpath '/home/codegyani/hive/student_details'
                                    into table stud_demo;
```

Create a partition table by using the following command: -

```
hive> create table student_part (id int, name string, age int, institute string)
        partitioned by (course string)
        row format delimited
        fields terminated by ',';
```

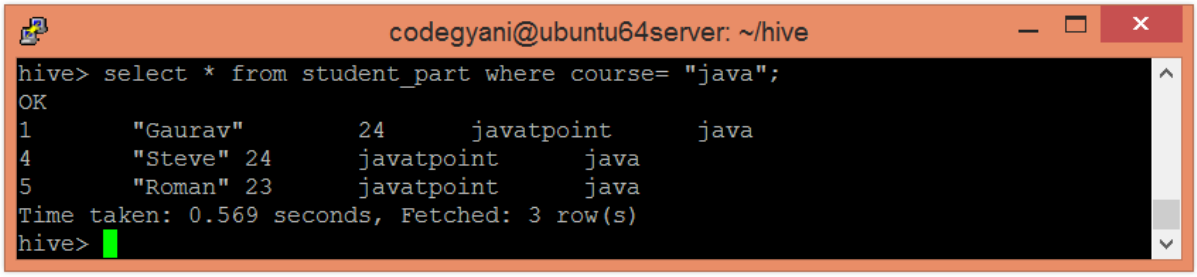
- Let's retrieve the entire data of the table by using the following command: -

```
hive> select * from student_part;
```



- Now, try to retrieve the data based on partitioned columns by using the following command: -

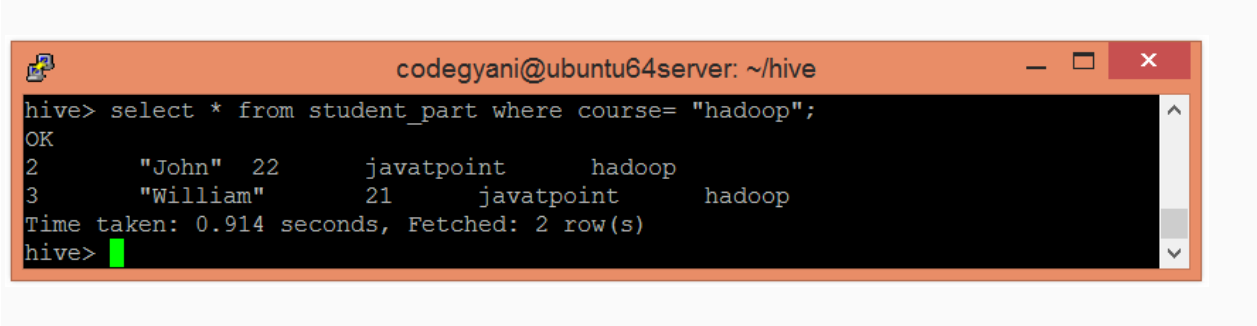
```
hive> select * from student_part where course= "java ";
```



In this case, we are not examining the entire data. Hence, this approach improves query response time.

- Let's also retrieve the data of another partitioned dataset by using the following command: -

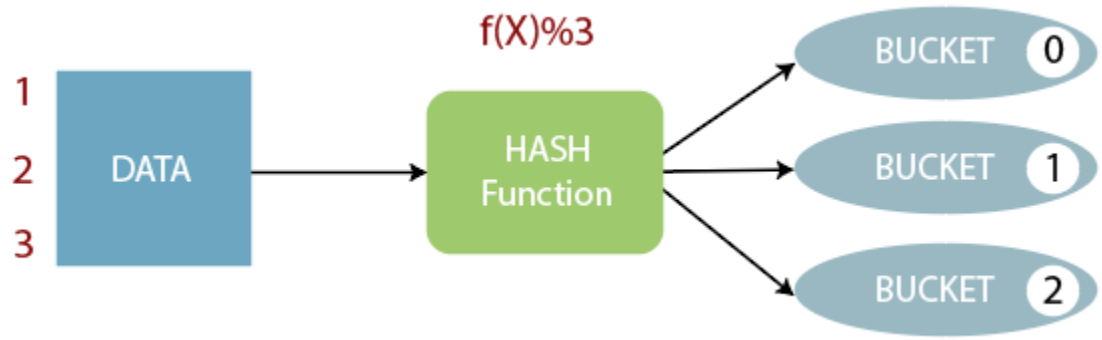
hive> select * from student_part where course= "hadoop";



Bucketing in Hive

The bucketing in Hive is a data organizing technique. It is similar to partitioning in Hive with an added functionality that it divides large datasets into more manageable parts known as buckets. So, we can use bucketing in Hive when the implementation of partitioning becomes difficult. However, we can also divide partitions further in buckets.

Working of Bucketing in Hive



- The concept of bucketing is based on the hashing technique.
- Here, modules of current column value and the number of required buckets is calculated (let say, F(x) % 3).
- Now, based on the resulted value, the data is stored into the corresponding bucket.

The Hive command for Bucketing is:

```
CREATE TABLE table_name
PARTITIONED BY (partition1 data_type, partition2 data_type,....)
CLUSTERED BY (column_name1, column_name2, ...)
SORTED BY (column_name [ASC|DESC], ...)] INTO num_bucketsBUCKETS;
```

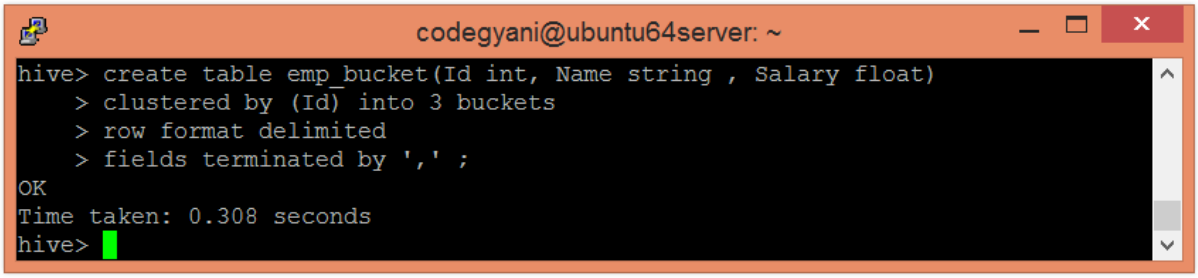
Example of Bucketing in Hive

- Enable the bucketing by using the following command: -

```
hive> set hive.enforce.bucketing = true;
```

- Create a bucketing table by using the following command: -

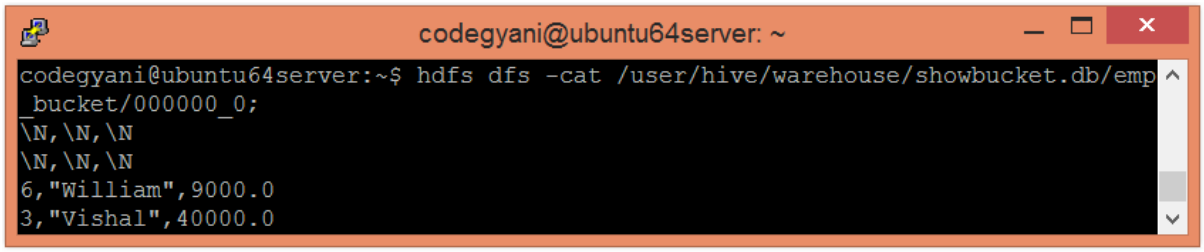
```
hive> create table emp_bucket(Id int, Name string , Salary float)
        clustered by (Id) into 3 buckets
        row format delimited
        fields terminated by ',' ;
```



- Now, insert the data of dummy table into the bucketed table.

```
hive> insert overwrite table emp_bucket select * from emp_demo;
```

Let's retrieve the data of bucket 0.



According to hash function :
6%3=0
3%3=0
So, these columns stored in bucket 0.

- Let's retrieve the data of bucket 1.

```
codegyani@ubuntu64server: ~
codegyani@ubuntu64server:~$ hdfs dfs -cat /user/hive/warehouse/showbucket.db/emp
_bucket/000001_0;
7,"Lisa",25000.0
4,"John",10000.0
1,"Gaurav",30000.0
```

According to hash function :
7%3=1
4%3=1
1%3=1

So, these columns stored in bucket 1.

- Let's retrieve the data of bucket 2.

```
codegyani@ubuntu64server: ~
codegyani@ubuntu64server:~$ hdfs dfs -cat /user/hive/warehouse/showbucket.db/emp
_bucket/000002_0;
8,"Ronit",20000.0
5,"Henry",25000.0
2,"Aryan",20000.0
```

According to hash function :
8%3=2
5%3=2
2%3=2
So, these columns stored in bucket 2.

Pros:

- It provides faster query response like partitioning.
- In bucketing due to equal volumes of data in each partition, joins at Map side will be quicker.

Cons:

- We can define a number of buckets during table creation. But loading of an equal volume of data has to be done manually by programmers.